

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0401

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I G. Yashmita(192511032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Annexure A

INDUSTRY PROBLEM SOLVING TASK

S.No	Industry Problem Solving Task Questions	Marks
1	A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.	5
2	A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm(FCFS,SSTF,SCAN) to reduce seektime and explain how it improves performance in this scenario.	5
3	An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.	5

4	A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.	5
5	A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Q1. Cloud Storage – File Allocation

Problem

A cloud storage company is experiencing delays during frequent file access. The system needs a file allocation method that provides **fast access, efficient storage utilization, and flexibility when files grow**.

Recommended Technique

Indexed allocation is recommended.

Comparison

- **Contiguous allocation:** Provides very fast access because blocks are adjacent, but suffers from external fragmentation and makes file expansion difficult.
- **Linked allocation:** Avoids external fragmentation and allows easy file growth, but random access is slow because blocks must be followed sequentially.
- **Indexed allocation:** Stores addresses of all file blocks in an index block, allowing direct access to required blocks.

Technical Justification

Indexed allocation is suitable for cloud storage because files can be stored in non-contiguous blocks while their locations are maintained in the index block. This provides efficient random access and allows files to grow without requiring a continuous free-space region.

Conclusion

Therefore, **Indexed Allocation** is the best choice for the cloud storage system because it provides **efficient random access, flexible file growth, and better handling of fragmented disk space**.

Q2. Data Center – Disk Scheduling

Problem

A data center is experiencing high disk-head movement while processing user requests. Excessive head movement increases **seek time and response time**, reducing overall system performance.

Recommended Technique

SSTF (Shortest Seek Time First) is recommended when the primary objective is to reduce average seek time.

Comparison

- **FCFS:** Simple and fair, but may cause large and unnecessary head movements.
- **SSTF:** Selects the request closest to the current head position, reducing average seek distance.
- **SCAN:** Moves the head in one direction and then reverses, providing better fairness but may involve more movement than SSTF for some workloads.

Technical Justification

SSTF continuously selects the nearest pending request. For example, if the head is at 50 and requests are 20, 55, 90, it selects 55 first because the movement is only 5 blocks. This reduces unnecessary movement and improves disk response time.

Conclusion

Therefore, **SSTF** is recommended because it generally provides **lower average seek time and reduced disk-head movement** compared with FCFS. However, SCAN may be preferred when starvation prevention and fairness are more important.

Q3. Bitmap – Free Space Management

Problem

The operating system uses a bitmap to manage free disk blocks, but allocation becomes slow as the disk size increases. Searching a very large bitmap bit-by-bit increases the allocation time.

Recommended Technique

A **hierarchical/grouped bitmap** is recommended to optimize free-space searching.

Comparison

- **Basic bitmap:** Simple and requires only one bit per disk block, but searching a large bitmap can be time-consuming.
- **Linked free-space list:** Can locate free blocks efficiently in some cases, but requires additional pointers and management overhead.
- **Hierarchical bitmap:** Divides the bitmap into groups and uses a summary structure to quickly identify groups containing free blocks.

Technical Justification

Instead of scanning the complete bitmap, the system first searches the summary bitmap to locate a group containing free blocks. It then searches only that group.

Summary Bitmap



Find free group



Search group



Allocate block

This reduces the number of bitmap entries that need to be examined and improves allocation performance for large disks.

Conclusion

Therefore, a **hierarchical bitmap** is recommended because it provides **faster free-block searching, lower allocation overhead, and better scalability for large storage systems**.

Q4. Real-Time Application – SCAN vs C-SCAN

Problem

A real-time application requires **consistent and predictable disk access times**. The scheduling algorithm should provide fair and relatively uniform waiting times for disk requests.

Recommended Technique

C-SCAN (Circular SCAN) is recommended.

Comparison

- **SCAN:** Moves the disk head in one direction and then reverses direction. It provides reasonable fairness but waiting times can vary depending on request location.
- **C-SCAN:** Services requests in one direction only. After reaching the end, it returns to the beginning and continues in the same direction.

SCAN:

→ → → → → ← ← ← ←

C-SCAN:

→ → → → → | → → → →



returns

Technical Justification

C-SCAN provides a more **uniform waiting time** because requests are serviced during a single directional sweep. This makes disk access more predictable, which is important for real-time applications.

Although C-SCAN may introduce additional head movement during the return to the beginning, it provides better consistency in request servicing.

Conclusion

Therefore, C-SCAN is more suitable for real-time applications because it provides **more predictable and uniform disk access times** compared with SCAN.

Q5. Hybrid Storage System – Allocation + Scheduling

Problem

A company wants to build a hybrid storage system for large-scale data access. The system needs both **efficient file allocation** and **reduced disk-head movement**.

Recommended Technique

The recommended combination is **Indexed Allocation + SSTF Scheduling**.

Comparison

File allocation:

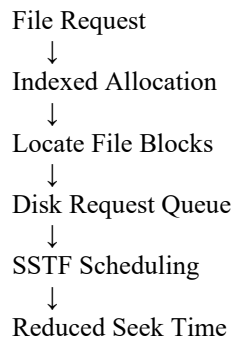
- **Contiguous:** Very fast access but suffers from fragmentation and difficult file expansion.
- **Linked:** Flexible and avoids external fragmentation but provides slower random access.
- **Indexed:** Provides direct access and supports files stored in non-contiguous blocks.

Disk scheduling:

- **FCFS:** Simple but can cause large head movements.
- **SSTF:** Selects the nearest request and reduces average seek time.
- **SCAN/C-SCAN:** Provides better fairness and predictability but may require additional head movement.

Technical Justification

Indexed allocation allows large files to be stored in different disk locations while an index block maintains their addresses. SSTF then selects the nearest pending disk request, reducing unnecessary head movement.



This combination provides **fast random file access, flexible storage, reduced average seek time, and efficient disk utilization.**

Conclusion

Therefore, **Indexed Allocation + SSTF** is a suitable combination for large-scale data access because it combines **efficient file access with reduced disk-head movement**. If predictable response time and fairness become more important, C-SCAN can be considered instead.